

DBI Dokumentation – Statify

Team

Dominik Nageler, Jonas Marte

1. Projektbeschreibung

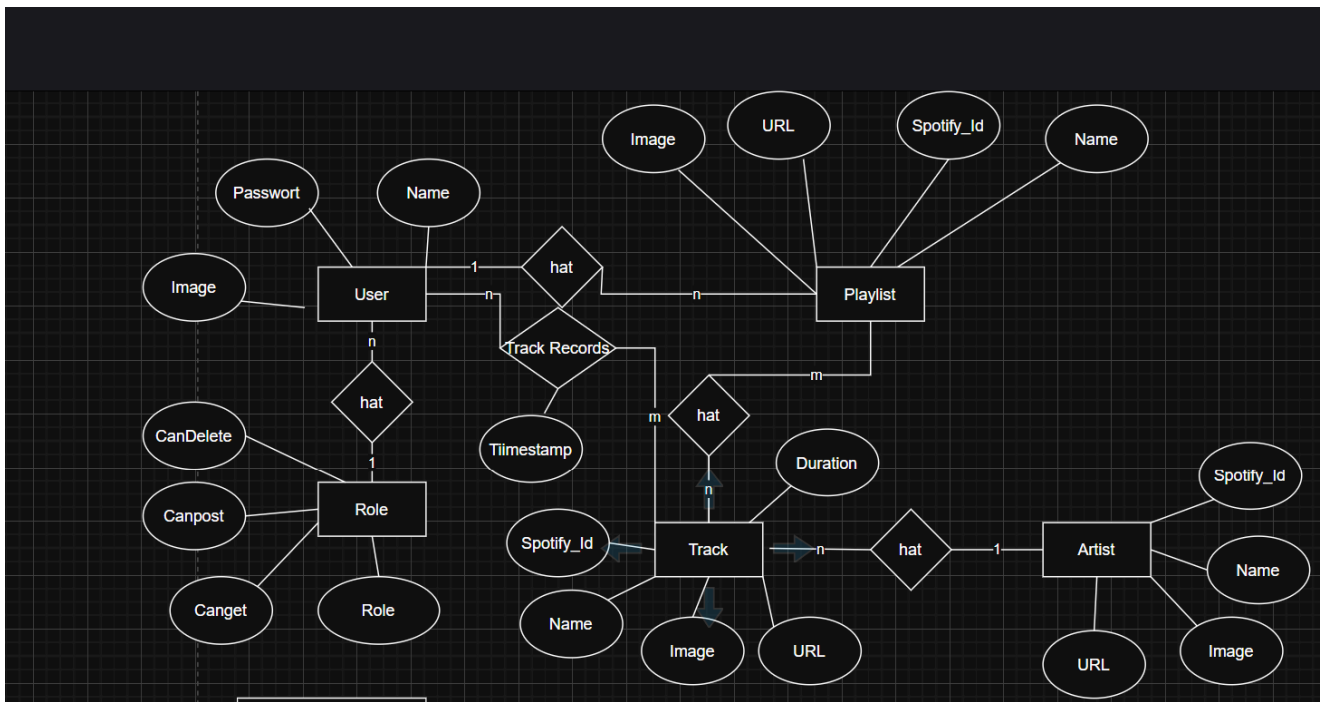
In unserer App soll man seine eigenen Spotify-Statistiken anschauen können. Dafür werden, falls vorhanden, die Daten aus unserer eigenen DB geladen. Die Daten werden bei jedem App-Start aktualisiert, sofern neue Daten vorhanden sind. So kann man nach längerer Benutzung Statistiken einsehen wie:

- Anzahl angehörter Minuten
- Meist gehörte/r
 - Artist
 - Track
 - Playlist

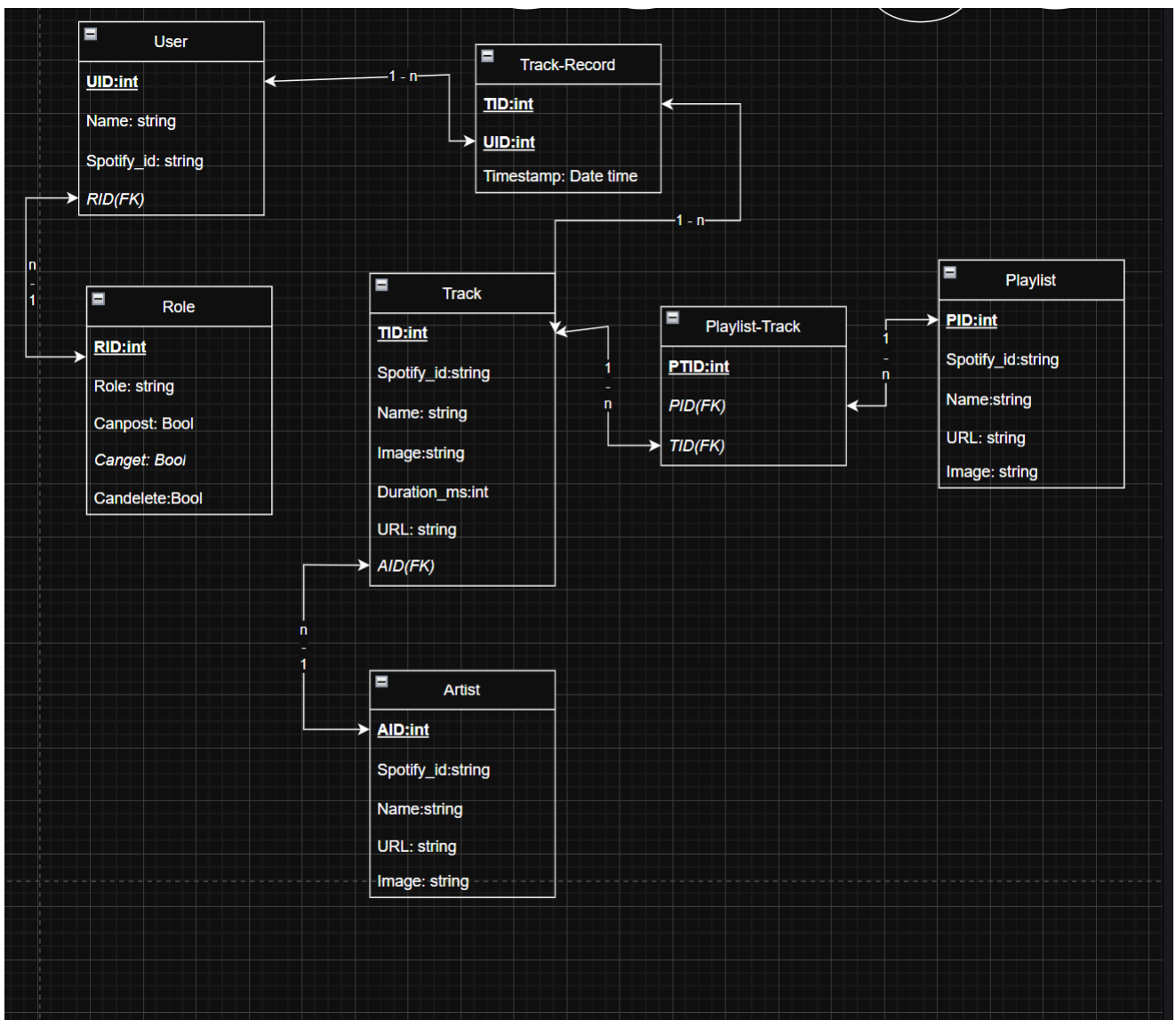
Im Mainwindow sieht man eine grobe Übersicht über ein paar spannende Daten. Über die Nav-Bar kann man genauere Daten anschauen, z. B. meistgehörte Artists, Tracks usw.

2. Planung

ERM



Relationales Modell



Normalformen

Alle Tabellen erfüllen die 1., 2. und 3. Normalform.

1NF

Jede Spalte enthält nur atomare (unteilbare) Werte, es gibt keine Wiederholungsgruppen und jede Zeile ist durch einen Primärschlüssel eindeutig identifizierbar. Alle unsere Tabellen sind so aufgebaut – z. B. speichert `Track_Record` pro Hörvorgang genau einen Timestamp und eine Duration, nicht eine Liste davon.

2NF

Alle Nicht-Schlüssel-Attribute sind voll funktional abhängig vom gesamten Primärschlüssel (relevant bei zusammengesetzten Primärschlüsseln).

3NF

Es gibt keine transitiven Abhängigkeiten zwischen Nicht-Schlüssel-Attributen. Zum Beispiel: `Follower_count` in `Artist` hängt direkt von `AID` ab, nicht von einem anderen Nicht-Schlüssel-Attribut. Berechnete Werte wie "Gesamtminuten" werden gar nicht gespeichert, sondern per SQL aus `Track_Record` abgeleitet.

3. Umsetzung

Technologien

Technologie	Version
Python	3.11
annotated-doc	0.0.4
annotated-types	0.7.0
anyio	4.13.0
click	8.3.3
colorama	0.4.6
fastapi	0.136.1
fastapi-restful	0.6.0
greenlet	3.5.0
h11	0.16.0
idna	3.13

Technologie	Version
mypy_extensions	1.1.0
psutil	5.9.8
pydantic	2.13.3
pydantic_core	2.46.3
SQLAlchemy	2.0.49
starlette	1.0.0
typing-inspect	0.9.0
typing-inspection	0.4.2
typing_extensions	4.15.0
uvicorn	0.46.0
bcrypt	4.0.1
spotipy	2.26.0
python-dotenv	1.2.2
passlib	1.7.4

Datenbank

User

Tabelle mit allen Usern.

Role

Tabelle für die Rollen, die an die User vergeben werden.

Track

Tabelle, um alle Tracks von der Spotify-API zu speichern.

Track_Record

Verbindungstabelle zwischen User und Track, speichert zusätzlich einen Timestamp.

Artist_Track

Verbindungstabelle zwischen Artist und Track.

Artist

Tabelle, um jeden Artist der abgespeicherten Tracks zu speichern.

Playlist_Track

Verbindungstabelle zwischen Playlist und Track.

Playlist

Tabelle, um Playlists zu speichern.

API-Endpunkte

Playlist (/playlist)

Methode	Pfad	Bedeutung	Auth
POST	/playlist/sync/{user_id}	Playlists von Spotify synchronisieren	admin
POST	/playlist/sync/{playlist_id}/tracks	Tracks einer Playlist von Spotify nachladen	admin

Methode	Pfad	Bedeutung	Auth
GET	/playlist/{playlist_id}/tracks	Tracks einer Playlist	user, admin
GET	/playlist/{user_id}	Alle Playlists eines Users	user, admin
DELETE	/playlist/{playlist_id}	Playlist löschen	admin

Track-Record (/track_record)

Methode	Pfad	Bedeutung	Auth
GET	/track_record/{user_id}	Hörverlauf mit Track-, Artist- und Playcount-Infos (JOIN)	user, admin
POST	/track_record/sync/{user_id}	Zuletzt gehörte Tracks von Spotify synchronisieren	admin

Artist (/artist)

Methode	Pfad	Bedeutung	Auth
GET	/artist/{user_id}	Artists eines Users mit Gesamt-Playtime (JOIN)	user, admin
DELETE	/artist/{artist_id}	Artist löschen	admin

User (/user)

Methode	Pfad	Bedeutung	Auth
POST	/user/register	Neuen User registrieren	öffentlich
POST	/user/login	User einloggen	öffentlich
PUT	/user/	User aktualisieren	user, admin
DELETE	/user/logout/{user_id}	User löschen / ausloggen	user, admin

Rollenmodell

User

Kann Post (mit Put) und Get.

Admin

Kann Post, Get und Delete.

4. Vergleich Planung ↔ Umsetzung

Geplant	Ergebnis	Bemerkung
Genre-Endpoint	ging nicht	Spotify-API-Endpoint zur Verfügung gestellt
User_Settings	keine Verwendung	Der Spotify-Token konnte über das Nugget-Package geholt werden (Tabelle war deshalb unnötig)
Kein Stats-Endpoint	Stats selbst aggregiert	Die Statistiken für die jeweiligen Models wurden in deren eigener Route abgefragt

5. KI-Unterstützung

Verwendete Tools

- ChatGPT
- Claude
- GitHub Copilot

Beispiel

Prompt:

```
hallo claude ich will mit fastapi ein usermodel erstellen und dort ein
passwort anlegen wie kann ich das hashen ?
```

Ergebnis:

```
Python

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)
```

```
def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

6. Projekttagbuch

Datum	Person	Tätigkeit
21.05.2026	Dominik	Repository, Grundstruktur, Projektbeschreibung
21.05.2026	Jonas	Datenbankplanung, erste ERM/RM-Struktur
22.05.2026	Jonas	ERM/RM erweitert, Tabellenstruktur geplant
24.05.2026	Dominik	Backend-Grundstruktur, requirements, FastAPI-Setup
25.05.2026	Jonas	Datenbankmodelle und Tabellenentwurf
26.05.2026	Dominik	User-/Settings-Struktur, erste API-Grundlagen
27.05.2026	Jonas	Track-/Genre-/Stats-Konzept vorbereitet
28.05.2026	Dominik	Artist-/Playlist-/User-Router vorbereitet
30.05.2026	Jonas	Track Records, Datenbanklogik, SQL-Abfragen
31.05.2026	Dominik	Artists und Playlists erweitert
01.06.2026	Jonas	Stats-Endpunkte, Aggregationen, JOIN-Abfragen
02.06.2026	Dominik	User- und Playlist-Endpunkte ergänzt
03.06.2026	Jonas	CRUD-Endpunkte, Validierung, Fehlerbehandlung
08.06.2026	Dominik	Rollenmodell, Auth-Grundlagen, API-Anpassungen
10.06.2026	Jonas	Statistikfunktionen, Top-Tracks/Top-Genres
10.06.2026	Dominik	Artist-/Playlist-Details, JOIN-Responses
11.06.2026	Jonas	Refactoring, SQL-Optimierung, Helper-Funktionen
11.06.2026	Dominik	Logging, Fehlerbehandlung, Router-Anpassungen
13.06.2026	Jonas	API-Tests, Bugfixes, Response-Modelle
13.06.2026	Dominik	Dokumentation, Projektbeschreibung, Endpunktübersicht
14.06.2026	Jonas	Finale DBI-Fixes, Statistik- und Datenbanklogik
14.06.2026	Dominik	Finale API-Anpassungen, Dokumentation ergänzt
15.06.2026	Dominik	Letzte Änderungen, Merge, Abgabevorbereitung

7. Bedienungsanleitung

Installation

```
pip install -r requirements.txt
```

Starten

```
uvicorn src.main:app --reload
```

API-Dokumentation

```
http://localhost:8888/docs
```